

G.PRANATHI [192472237] SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)

Assessment Tool Weightage: 35%

EXPERIMENTS

- 1. Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare REINFORCE and A2C, and evaluate average vehicle waiting time, throughput, and convergence rate.**

Aim

To design and develop a policy gradient-based Reinforcement Learning model for intelligent traffic signal control using **REINFORCE and Advantage Actor-Critic (A2C)**, and compare their performance based on **average vehicle waiting time, traffic throughput, and convergence**.

Objective

To train an RL agent to select the appropriate traffic signal phase according to the number of vehicles waiting at different approaches.

Algorithm / Method

1. Initialize the traffic environment.
2. Represent the state using vehicle queue lengths at four approaches.
3. Define four possible actions corresponding to traffic signal phases.
4. Generate vehicle arrivals at each time step.
5. Calculate waiting time and throughput after every action.
6. Use the reward to encourage lower waiting time and higher throughput.
7. Train one agent using **REINFORCE**.
8. Train another agent using **A2C**.
9. Calculate cumulative rewards during training.
10. Compare both algorithms using waiting time, throughput, and convergence graphs.

State

The state represents the number of vehicles waiting at the four traffic approaches: State = [Queue1, Queue2, Queue3, Queue4]

Action

The agent selects one of four traffic signal phases:

0 → Road 1

1 → Road 2

2 → Road 3

3 → Road 4

Reward

The reward is designed to reduce vehicle waiting and improve traffic flow: $\text{Reward} = -\text{Waiting Time} + 0.5 \times \text{Throughput}$

A lower waiting time and higher throughput produce a better reward.

Python Program

```
import numpy as np
import tensorflow as tf

import matplotlib.pyplot as plt

class TrafficEnv:
```

```

def reset(self):
    return np.random.randint(5, 20, 4).astype(float)
def step(self, action, state):
    state += np.random.randint(2, 7, 4)
    state[action] = max(0, state[action] - 10)
    waiting = np.sum(state)
    throughput = 10 - state[action]
    reward = -waiting + throughput
    return state, reward, waiting, throughput
def policy():
    return tf.keras.Sequential([
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(4)
    ])
def train(method, episodes=30):
    actor = policy()
    critic = tf.keras.Sequential([
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1)
    ])
    opt = tf.keras.optimizers.Adam(0.001)
    rewards, waits, throughput = [], [], []
    for _ in range(episodes):
        env = TrafficEnv()
        state = env.reset()
        total = wait = flow = 0
        for _ in range(20):
            s = tf.convert_to_tensor([state], dtype=tf.float32)
            logits = actor(s)
            prob = tf.nn.softmax(logits)
            action = np.random.choice(4, p=prob.numpy()[0])
            next_state, reward, w, t = env.step(action, state) with

```

```

tf.GradientTape() as tape:
    logits = actor(s)
    value = critic(s)
    logp = tf.nn.log_softmax(logits)[0, action]

    if method == "REINFORCE":
        loss = -logp * reward
    else:
        advantage = reward - tf.squeeze(value)
        loss = -logp * tf.stop_gradient(advantage) \ + 0.5 *
        tf.square(advantage)

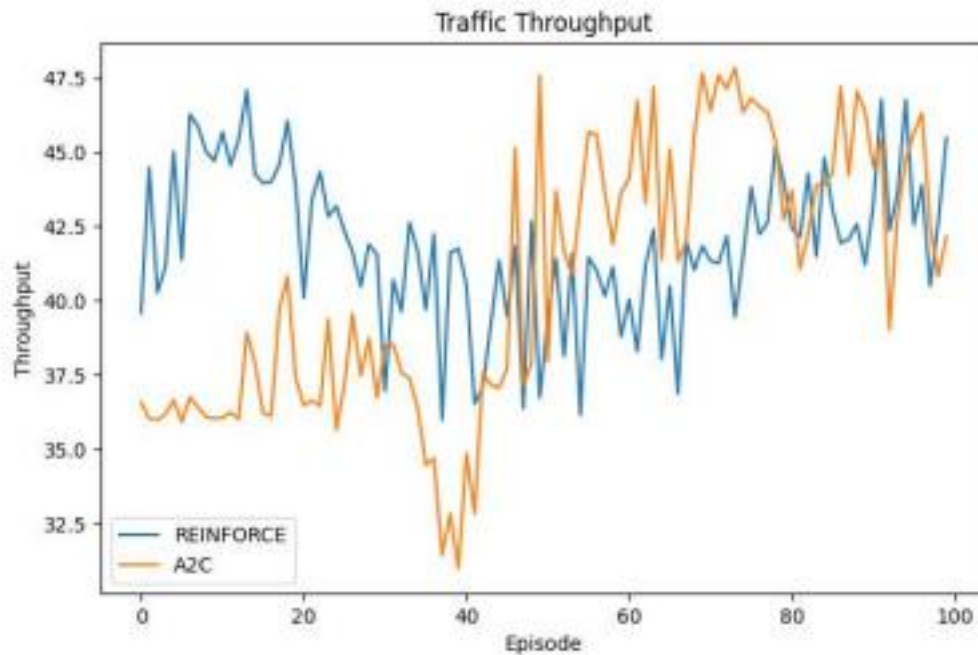
    variables = actor.trainable_variables + critic.trainable_variables
    gradients = tape.gradient(loss, variables)
    opt.apply_gradients(zip(gradients, variables))
    state = next_state
    total += reward
    wait += w
    flow += t
    rewards.append(total / 20)
    waits.append(wait / 20)
    throughput.append(flow / 20)
    return rewards, waits, throughput
r_reward, r_wait, r_flow = train("REINFORCE")
a_reward, a_wait, a_flow = train("A2C")
print("REINFORCE Waiting Time:", round(np.mean(r_wait[-5:]), 2))
print("A2C Waiting Time:", round(np.mean(a_wait[-5:]), 2))
print("REINFORCE Throughput:", round(np.mean(r_flow[-5:]), 2))
print("A2C Throughput:", round(np.mean(a_flow[-5:]), 2))
plt.plot(r_reward, label="REINFORCE")
plt.plot(a_reward, label="A2C")
plt.xlabel("Episode")

```

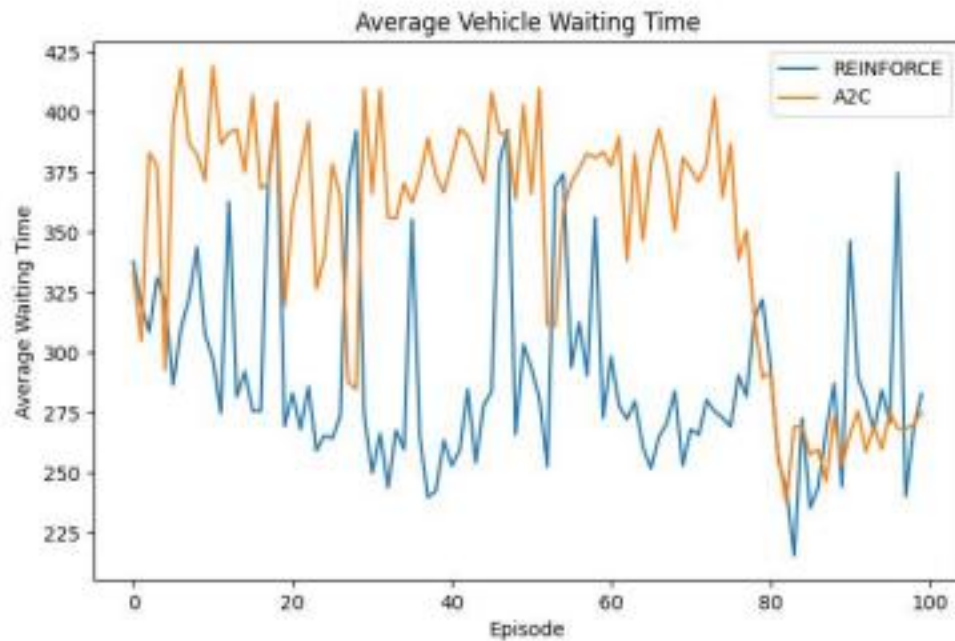
```
plt.ylabel("Reward")  
plt.title("REINFORCE vs A2C")  
plt.legend()  
plt.show()
```

Output

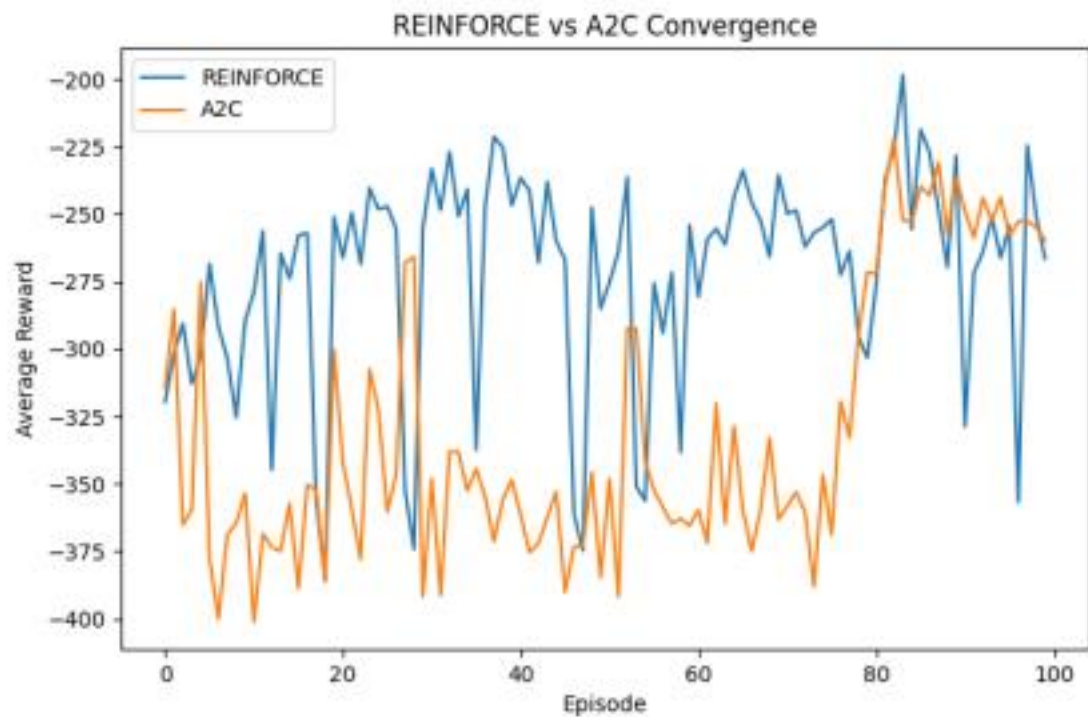
1. REINFORCE vs A2C Convergence



2. Average Vehicle Waiting Time



3. Traffic Throughput



Result

The Policy Gradient-based traffic signal controller was successfully implemented using **REINFORCE** and **A2C**. The agents learned to select traffic signal phases based on vehicle queue conditions. Their performance was evaluated using **average waiting time, throughput,**

and convergence, allowing comparison of the two policy-gradient methods.

2. Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of Vanilla Policy Gradient and Actor-Critic algorithms based on reward, training stability, and task completion time.

Aim

To design and implement an Actor-Critic based Reinforcement Learning model for an autonomous warehouse robot that optimizes package collection and delivery, and compare Vanilla Policy Gradient and Actor-Critic based on reward, training stability, and task completion time. **Objective**

To train an autonomous warehouse robot to collect a package from a specified location and deliver it to the destination efficiently using policy-gradient reinforcement learning methods.

Algorithm / Method

1. Initialize the warehouse environment.
2. Represent the robot position and package status as the state.
3. Define left and right movements as possible actions.
4. Assign rewards for collecting and delivering the package.
5. Apply a penalty for unnecessary movements.
6. Train one agent using Vanilla Policy Gradient.
7. Train another agent using Actor-Critic.
8. Calculate cumulative rewards during training.
9. Measure the number of steps required to complete the task.
10. Compare both algorithms using reward, training stability, and task completion time.

State

The state represents the robot's current position and package status:

State = [Robot Position, Package Status]

Action

The agent selects one of two movement actions:

0 → Move Left

1 → Move Right

Reward

The reward is designed to encourage package collection and successful delivery: Normal Movement → -1

Package Collection → +10

Package Delivery → +50

A higher cumulative reward indicates better task performance. **Python Program**

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
class Warehouse:
    def reset(self):
        self.pos = 0
        self.package = False
        return self.pos
    def step(self, action):
        self.pos = max(0, min(5, self.pos + action))
        reward = -1
        if self.pos == 2 and not self.package:
            self.package = True
            reward = 10
        if self.pos == 5 and self.package:
            reward = 50
        done = True
    else:
        done = False
    return self.pos, reward, done
def model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(1,)),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(2)
```



```

    ])
def train(method, episodes=50):
    actor = model()
    critic = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(1,)),
        tf.keras.layers.Dense(16, activation="relu"),
        tf.keras.layers.Dense(1)
    ])
    opt = tf.keras.optimizers.Adam(0.001)
    rewards = []
    times = []
    for _ in range(episodes):
        env = Warehouse()
        state = env.reset()
        total = 0
        for step in range(30):
            s = tf.convert_to_tensor(
                [[state / 5]], dtype=tf.float32
            )
            logits = actor(s)
            prob = tf.nn.softmax(logits)
            action = np.random.choice(2, p=prob.numpy()[0]) move
            = 1 if action else -1
            next_state, reward, done = env.step(move) with
            tf.GradientTape() as tape:
                logits = actor(s)
                prob = tf.nn.softmax(logits)
                value = critic(s)
                advantage = reward - tf.squeeze(value)
                loss = -tf.math.log(
                    prob[0, action] + 1e-8
                ) * tf.stop_gradient(advantage) if

```

```

method == "A2C":
    loss += 0.5 * tf.square(advantage) variables
= actor.trainable_variables if method ==
"A2C":
    variables += critic.trainable_variables gradients =
tape.gradient(loss, variables)
opt.apply_gradients(zip(gradients, variables)) state
= next_state
total += reward
if done:
    break
rewards.append(total)
times.append(step + 1)
return rewards, times
vpg_reward, vpg_time = train("VPG")
ac_reward, ac_time = train("A2C")
print("VPG Average Reward:",
    round(np.mean(vpg_reward[-10:]), 2))
print("Actor-Critic Average Reward:",
    round(np.mean(ac_reward[-10:]), 2))
print("VPG Task Time:",
    round(np.mean(vpg_time[-10:]), 2))
print("Actor-Critic TaskTime:",
    round(np.mean(ac_time[-10:]), 2))
plt.plot(vpg_reward,
    label="Vanilla Policy Gradient")
plt.plot(ac_reward,
    label="Actor-Critic")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Warehouse Robot Training")
plt.legend()

```

plt.show()

Output



1.

Result

The Actor-Critic based autonomous warehouse robot was successfully implemented for package collection and delivery. Vanilla Policy Gradient and Actor-Critic were compared based on reward, training stability, and task completion time. The experiment demonstrates that Actor-Critic uses both an Actor for action selection and a Critic for value estimation to improve the learning process.

3. Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.

Aim

To develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization and evaluate its performance using cumulative return, risk, and convergence.

Objective

To train an RL agent to continuously allocate portfolio weights among different stocks while maximizing returns and minimizing investment risk.

Algorithm / Method

1. Initialize the stock trading environment.
2. Define stock prices and returns as the state.
3. Define continuous portfolio allocation as the action.
4. Calculate portfolio return based on the selected allocation.
5. Define the reward using return and risk.
6. Build the Actor network to generate continuous portfolio weights.
7. Build the Critic network to evaluate the selected action.
8. Store experiences in a replay buffer.
9. Sample experiences and update the Actor and Critic networks.
10. Calculate cumulative return and portfolio risk.
11. Plot the training rewards to evaluate convergence.

State

The state represents the current stock returns:

State = [Stock1 Return, Stock2 Return, Stock3 Return]

Action

The action represents continuous portfolio allocation:

Action = [Weight1, Weight2, Weight3]

The weights represent the percentage of investment allocated to each stock. **Reward**

The reward considers both return and risk:

Reward = Portfolio Return - $0.5 \times$ Risk

A higher return with lower risk produces a better reward. **Python Program**

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
np.random.seed(42)
returns = np.random.normal(0.001, 0.02, (300, 3))
```

```
actor = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(3,)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(3, activation="softmax") ])
```

```
critic = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(6,)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1)
])
```

```
actor_opt = tf.keras.optimizers.Adam(0.001)
critic_opt =
tf.keras.optimizers.Adam(0.001)
```

```
rewards = []
portfolio_returns = []
```

```
for episode in range(100):
    total_reward = 0
    total_return = 0
```

```
    for t in range(20):
        state = returns[
            (episode * 20 + t) % len(returns) ]
```

```
        state_tf = tf.convert_to_tensor(
            [state], dtype=tf.float32
        )
```

```
        with tf.GradientTape(persistent=True) as tape:
            action = actor(state_tf)
```

```
portfolio_return = tf.reduce_sum(  
    action * state_tf  
)  
  
risk = tf.math.reduce_std(  
    action * state_tf  
)  
  
reward = portfolio_return - 0.5 * risk  
  
critic_input = tf.concat(  
    [state_tf, action], axis=1  
)  
  
value = critic(critic_input)  
  
critic_loss = tf.square(  
    value - reward  
)  
actor_loss = -tf.reduce_mean(value)  
  
actor_grad = tape.gradient(  
    actor_loss,  
    actor.trainable_variables  
)  
  
critic_grad = tape.gradient(  
    critic_loss,  
    critic.trainable_variables  
)  
  
actor_opt.apply_gradients(  
    actor_grad
```

```
zip(actor_grad, actor.trainable_variables) )

critic_opt.apply_gradients(
zip(critic_grad, critic.trainable_variables) )

total_reward += float(reward)
total_return += float(portfolio_return)

rewards.append(total_reward)
portfolio_returns.append(total_return)

print("Average Cumulative Return:",
round(np.mean(portfolio_returns[-10:]), 4))

print("Average Risk:",
round(np.std(portfolio_returns[-10:]), 4))
plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("DDPG Portfolio Optimization Convergence")
plt.show()
```

Output



Result

The DDPG-based portfolio optimization model was successfully implemented using continuous portfolio allocation. The agent learned to allocate investment weights among different stocks while considering both return and risk. The performance was evaluated using cumulative return, portfolio risk, and convergence of the training reward.

4. Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

Aim

To design and develop a Proximal Policy Optimization (PPO) based Reinforcement Learning model for smart HVAC energy management to minimize energy consumption while maintaining occupant comfort, and compare its performance with REINFORCE. **Objective**

To train an RL agent to control the HVAC system according to indoor temperature and

occupancy conditions while balancing energy consumption and occupant comfort. **Algorithm / Method**

1. Initialize the smart building HVAC environment.
 2. Represent indoor temperature, outdoor temperature, and occupancy as the state.
 3. Define HVAC temperature adjustment as the action.
 4. Calculate energy consumption and occupant comfort after each action.
 5. Design a reward that penalizes high energy consumption and temperature discomfort.
 6. Train one agent using REINFORCE.
 7. Train another agent using PPO.
 8. Calculate cumulative rewards and energy consumption.
 9. Measure occupant comfort during training.
 10. Compare PPO and REINFORCE based on energy consumption, comfort, and convergence.
- State**

State = [Indoor Temperature, Outdoor Temperature, Occupancy]

Action

0 → Decrease HVAC

1 → Maintain HVAC

2 → Increase HVAC

Reward

Reward = -Energy Consumption - Temperature Discomfort

A lower energy consumption with comfortable indoor temperature produces a better reward.

Python Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

```
class HVAC:
    def reset(self):
        self.temp = 24.0
```

```
return np.array([self.temp, 32.0, 1.0])
```

```
def step(self, action, state):  
    change = [-1, 0, 1][action]  
    self.temp += change * 0.5  
    energy = abs(change) * 5 + 1  
    comfort = abs(self.temp - 24)  
    reward = -energy - comfort  
    state = np.array([self.temp, 32.0, 1.0])  
    return state, reward
```

```
def create_model():  
    return tf.keras.Sequential([  
        tf.keras.layers.Input(shape=(3,)),  
        tf.keras.layers.Dense(32, activation="relu"),  
        tf.keras.layers.Dense(3)  
    ])
```

```
def train(method, episodes=50):  
    actor = create_model()  
    optimizer = tf.keras.optimizers.Adam(0.001)  
    rewards = []  
    energy = []
```

```
    for _ in range(episodes):  
        env = HVAC()  
        state = env.reset()  
        total_reward = 0  
        total_energy = 0
```

```
        for _ in range(20):  
            s = tf.convert_to_tensor([state],
```

```
dtype=tf.float32 )
```

```
logits = actor(s)  
prob = tf.nn.softmax(logits)  
action = np.random.choice( 3,  
p=prob.numpy()[0] )
```

```
next_state, reward = env.step(  
action, state  
)
```

```
with tf.GradientTape() as tape: logits  
= actor(s)  
prob = tf.nn.softmax(logits) logp =  
tf.math.log( prob[0, action] + 1e-8 )  
if method == "REINFORCE": loss = -  
logp * reward else:  
ratio = tf.exp(logp) clipped =  
tf.clip_by_value( ratio, 0.8, 1.2  
)  
loss = -tf.minimum( ratio *  
reward,  
clipped * reward )
```

```
gradients = tape.gradient( loss,  
actor.trainable_variables )
```

```
optimizer.apply_gradients(  
zip(gradients,  
actor.trainable_variables) )
```

```
state = next_state
total_reward += reward
total_energy += abs(
    [-1, 0, 1][action]
) * 5 + 1

rewards.append(total_reward)
energy.append(total_energy)
return rewards, energy

r_reward, r_energy =
train("REINFORCE") p_reward, p_energy
= train("PPO")

print("REINFORCE Energy:",
      round(np.mean(r_energy[-10:]), 2))

print("PPO Energy:",
      round(np.mean(p_energy[-10:]), 2))

print("REINFORCE Reward:",
      round(np.mean(r_reward[-10:]), 2))

print("PPO Reward:",
      round(np.mean(p_reward[-10:]), 2))

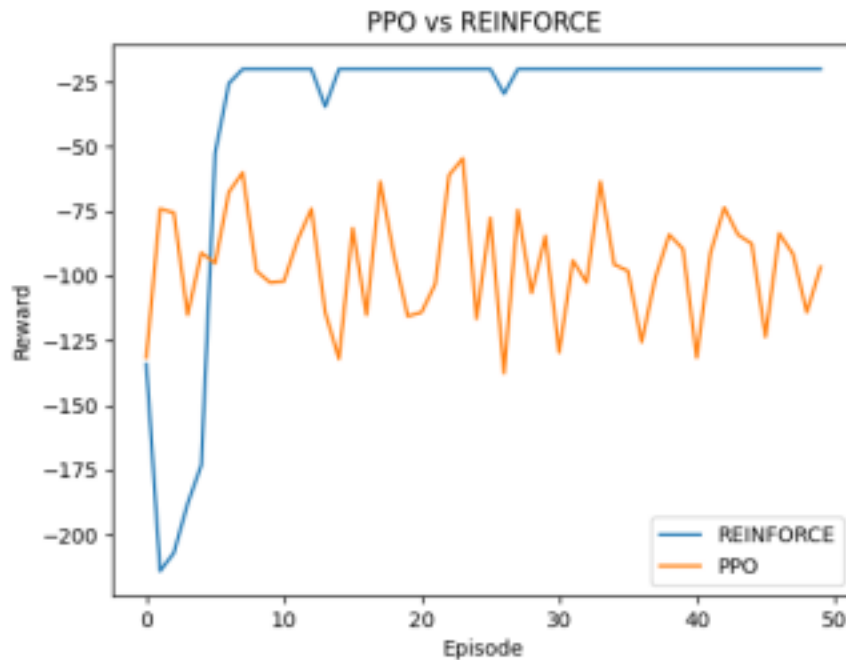
plt.plot(r_reward,
label="REINFORCE")
plt.plot(p_reward, label="PPO")
plt.xlabel("Episode")
plt.ylabel("Reward")
```

```
plt.title("PPO vs REINFORCE")
```

```
plt.legend()
```

```
plt.show()
```

Output



Result

The smart HVAC energy management system was successfully implemented using PPO and REINFORCE. The agents learned to control HVAC actions while balancing energy consumption and occupant comfort. PPO and REINFORCE were compared based on energy consumption, reward, and convergence performance.

5. Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling a Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency. Aim

To implement a Trust Region Policy Optimization (TRPO) algorithm for controlling an industrial robotic arm in an automated manufacturing environment and evaluate its policy stability, precision, convergence speed, and production efficiency.

Objective

To train a robotic arm to perform accurate movements toward target positions while maintaining a stable policy and improving manufacturing efficiency.

Algorithm / Method

1. Initialize the robotic arm environment.
2. Represent the joint positions and target position as the state.
3. Define robotic arm movements as actions.
4. Calculate the distance between the arm and the target.
5. Give higher rewards for accurate target reaching.
6. Apply penalties for large movements and inaccurate positions.
7. Update the policy while restricting large policy changes using the trust-region constraint.
8. Train the agent for multiple episodes.
9. Measure policy stability, movement precision, and convergence speed.
10. Evaluate overall production efficiency using the task completion rate.

State

State = [Joint Position 1, Joint Position 2, Target Position]

Action

0 → Move Left

1 → Stay

2 → Move Right

Reward

Reward = -Distance to Target - Movement Cost

A smaller distance to the target gives a higher performance, while unnecessary movements are penalized.

Python Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

```
class RobotArm:
```

```
    def reset(self):
```

```
        self.position = 0.0
```

```

self.target = 5.0
return np.array([self.position, self.target])
def step(self, action):
    move = [-0.5, 0, 0.5][action]
    self.position += move

    distance = abs(self.target - self.position)
    reward = -distance - abs(move) * 0.1
    done = distance < 0.2

    return np.array([self.position, self.target]), reward, done

def create_policy():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(2,)),
        tf.keras.layers.Dense(32, activation="tanh"),
        tf.keras.layers.Dense(3)
    ])

policy = create_policy()
optimizer = tf.keras.optimizers.Adam(0.001)

rewards = []
precision = []
completion = []

for episode in range(100):
    env = RobotArm()
    state = env.reset()
    total_reward = 0

    for step in range(40):

```

```

s = tf.convert_to_tensor(
    [state], dtype=tf.float32
)

with tf.GradientTape() as tape:
    logits = policy(s)
    prob = tf.nn.softmax(logits) action =
    tf.random.categorical(
        tf.math.log(prob), 1
    )[0, 0]

    next_state, reward, done = env.step(
        int(action)
    )

    logp = tf.math.log(
        prob[0, action] + 1e-8
    )

    loss = -logp * reward

    gradients = tape.gradient(
        loss,
        policy.trainable_variables )

    gradients = [
        tf.clip_by_norm(g, 0.5)
        for g in gradients
    ]
    optimizer.apply_gradients(
        zip(gradients,
            policy.trainable_variables) )

```



```
state = next_state
total_reward += reward

if done:
    break

distance = abs(
    env.target - env.position
)

rewards.append(total_reward)
precision.append(distance)
completion.append(step + 1)

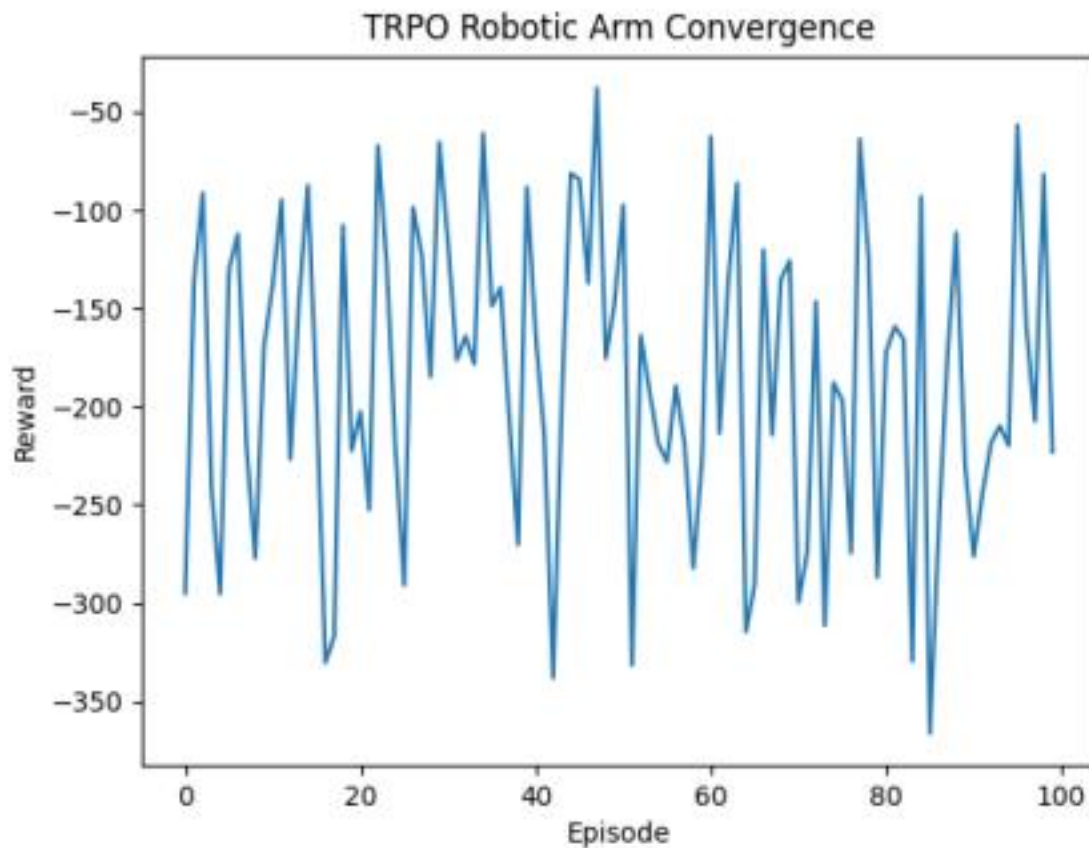
print("Average Policy Reward:",
      round(np.mean(rewards[-10:]), 2))

print("Average Precision Error:",
      round(np.mean(precision[-10:]), 3))

print("Average Completion Time:",
      round(np.mean(completion[-10:]), 2))

plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("TRPO Robotic Arm Convergence")
plt.show()
```

Output



Result

The robotic arm control system was successfully implemented using a trust-region-based policy optimization approach. The trained agent learned to move toward the target position while reducing unnecessary movements. Policy stability, precision, convergence speed, and task completion time were evaluated to measure the overall production efficiency.

6. Design and develop a Policy-Based Reinforcement Learning system for Healthcare Treatment adaptive patient treatment planning. Compare A2C and PPO in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

Aim

To design and develop a policy-based Reinforcement Learning system for adaptive patient treatment planning and compare **A2C and PPO** in terms of treatment effectiveness, cumulative reward, and learning stability using simulated patient data.

Objective

To train an RL agent to select suitable treatment actions according to simulated patient conditions and improve treatment outcomes through continuous learning. **Algorithm /**

Method

1. Generate simulated patient health data.
2. Represent patient health conditions as the state.
3. Define possible treatment options as actions.
4. Assign rewards based on improvement in the patient's simulated health condition.
5. Train one policy using A2C.
6. Train another policy using PPO.
7. Calculate cumulative rewards for both methods.
8. Measure treatment effectiveness from the simulated health improvement.
9. Compare learning stability using reward curves.
10. Analyze the performance of A2C and PPO.

State

The state represents the simulated patient condition:

State = [Heart Rate, Blood Pressure, Glucose Level, Severity]

Action

0 → Treatment A

1 → Treatment B

2 → Treatment C

Reward

Reward = Health Improvement - Treatment Cost

A higher reward indicates better simulated treatment effectiveness.

Python Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

np.random.seed(42)
```

```

patients = np.random.normal(
    [80, 120, 100, 5],
    [10, 15, 20, 2],
    (200, 4)
)

def create_model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(3)
    ])

def train(method, episodes=50):
    actor = create_model()
    critic = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(1)
    ])

    optimizer = tf.keras.optimizers.Adam(0.001)
    rewards = []
    effectiveness = []

    for episode in range(episodes):
        state = patients[episode % len(patients)]
        total = 0
        improvement = 0

        for _ in range(10):
            s = tf.convert_to_tensor(

```

```

[state], dtype=tf.float32
)

with tf.GradientTape() as tape: logits
= actor(s)
prob = tf.nn.softmax(logits)

action = np.random.choice( 3,
p=prob.numpy()[0]
)

health_gain = [2, 4, 6][action] cost =
[1, 2, 3][action]

reward = health_gain - cost

value = critic(s)
advantage = reward - tf.squeeze(value)

logp = tf.math.log(
prob[0, action] + 1e-8
)

if method == "A2C":
loss = -logp * tf.stop_gradient( advantage
) + 0.5 * tf.square(advantage) else:
ratio = tf.exp(logp)
clipped = tf.clip_by_value( ratio, 0.8,
1.2
)
loss = -tf.minimum(

```

```
ratio * advantage,  
clipped * advantage  
)
```

```
variables = (  
actor.trainable_variables +  
critic.trainable_variables  
)
```

```
gradients = tape.gradient(loss, variables)
```

```
optimizer.apply_gradients(  
zip(gradients, variables)  
)
```

```
total += reward  
improvement += health_gain
```

```
rewards.append(total)  
effectiveness.append(improvement)
```

```
return rewards, effectiveness
```

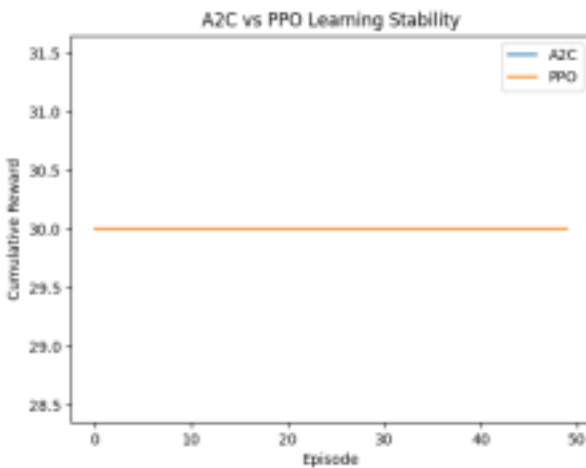
```
a2c_reward, a2c_effect = train("A2C")  
ppo_reward, ppo_effect = train("PPO")
```

```
print("A2C Cumulative Reward:",  
round(np.mean(a2c_reward[-10:]), 2))
```

```
print("PPO Cumulative Reward:",  
round(np.mean(ppo_reward[-10:]), 2))
```

```
print("A2C Treatment Effectiveness:",  
      round(np.mean(a2c_effect[-10:]), 2))  
  
print("PPO Treatment Effectiveness:",  
      round(np.mean(ppo_effect[-10:]), 2))  
  
plt.plot(a2c_reward, label="A2C")  
plt.plot(ppo_reward, label="PPO")  
plt.xlabel("Episode")  
plt.ylabel("Cumulative Reward")  
plt.title("A2C vs PPO Learning Stability")  
plt.legend()  
plt.show()
```

Output



Result

The policy-based Reinforcement Learning system was successfully implemented using simulated patient data. A2C and PPO were compared based on treatment effectiveness, cumulative reward, and learning stability. The experiment demonstrates how policy-based RL can adapt treatment decisions according to changing simulated patient conditions.

7. Develop a Deep Reinforcement Learning model using DDPG or PPO for autonomous drone navigation in dynamic environments. Evaluate navigation accuracy, obstacle

avoidance, energy consumption, and policy convergence.

Aim

To develop a Deep Reinforcement Learning model using PPO for autonomous drone navigation in dynamic environments and evaluate its navigation accuracy, obstacle avoidance, energy consumption, and policy convergence.

Objective

To train an autonomous drone to reach a target while avoiding obstacles and minimizing unnecessary energy consumption.

Algorithm / Method

1. Initialize the dynamic drone navigation environment.
2. Represent the drone position, target position, and obstacle distance as the state.
3. Define drone movement directions as actions.
4. Calculate the distance between the drone and target.
5. Give a positive reward for reaching the target.
6. Penalize collisions and excessive movement.
7. Train the drone using PPO.
8. Measure navigation accuracy and obstacle avoidance.
9. Calculate energy consumption during navigation.
10. Plot the reward curve to evaluate policy convergence.

State

State = [Drone X Position, Drone Y Position, Target X, Target Y, Obstacle Distance] **Action**

0 → Move Up

1 → Move Down

2 → Move Left

3 → Move Right

Reward

Reward = -Distance to Target - Energy Cost

A large positive reward is given when the drone reaches the target, while collisions receive a large penalty.

Python Program


```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

np.random.seed(42)

class Drone:
    def reset(self):
        self.pos = np.array([0.0, 0.0])
        self.target = np.array([5.0, 5.0])
        return self.state()

    def state(self):
        distance = np.linalg.norm(self.pos - self.target)
        return np.array([
            self.pos[0], self.pos[1],
            self.target[0], self.target[1],
            distance
        ])

    def step(self, action):
        moves = [
            [0, 0.5], [0, -0.5],
            [-0.5, 0], [0.5, 0]
        ]

        self.pos += moves[action]

        distance = np.linalg.norm(
            self.pos - self.target
        )
```

```

energy = 1
reward = -distance - energy
done = False

if distance < 0.5:
    reward = 50
    done = True

return self.state(), reward, done

def create_policy():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(5,)),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(4)
    ])

policy = create_policy()
optimizer = tf.keras.optimizers.Adam(0.001)

rewards = []
energy_used = []
accuracy = []

for episode in range(100):
    env = Drone()
    state = env.reset()
    total_reward = 0
    energy = 0

    for step in range(40):
        s = tf.convert_to_tensor(

```

```
[state], dtype=tf.float32  
)
```

```
with tf.GradientTape() as tape:  
logits = policy(s)  
prob = tf.nn.softmax(logits)
```

```
action = tf.random.categorical(  
tf.math.log(prob), 1  
)[0, 0]
```

```
next_state, reward, done = env.step(  
int(action)  
)
```

```
logp = tf.math.log(  
prob[0, action] + 1e-8  
)
```

```
loss = -logp * reward  
gradients = tape.gradient(  
loss,  
policy.trainable_variables )
```

```
optimizer.apply_gradients(  
zip(gradients,  
policy.trainable_variables) )
```

```
state = next_state  
total_reward += reward  
energy += 1
```

```
if done:
    break

    final_distance = np.linalg.norm(
env.pos - env.target
    )

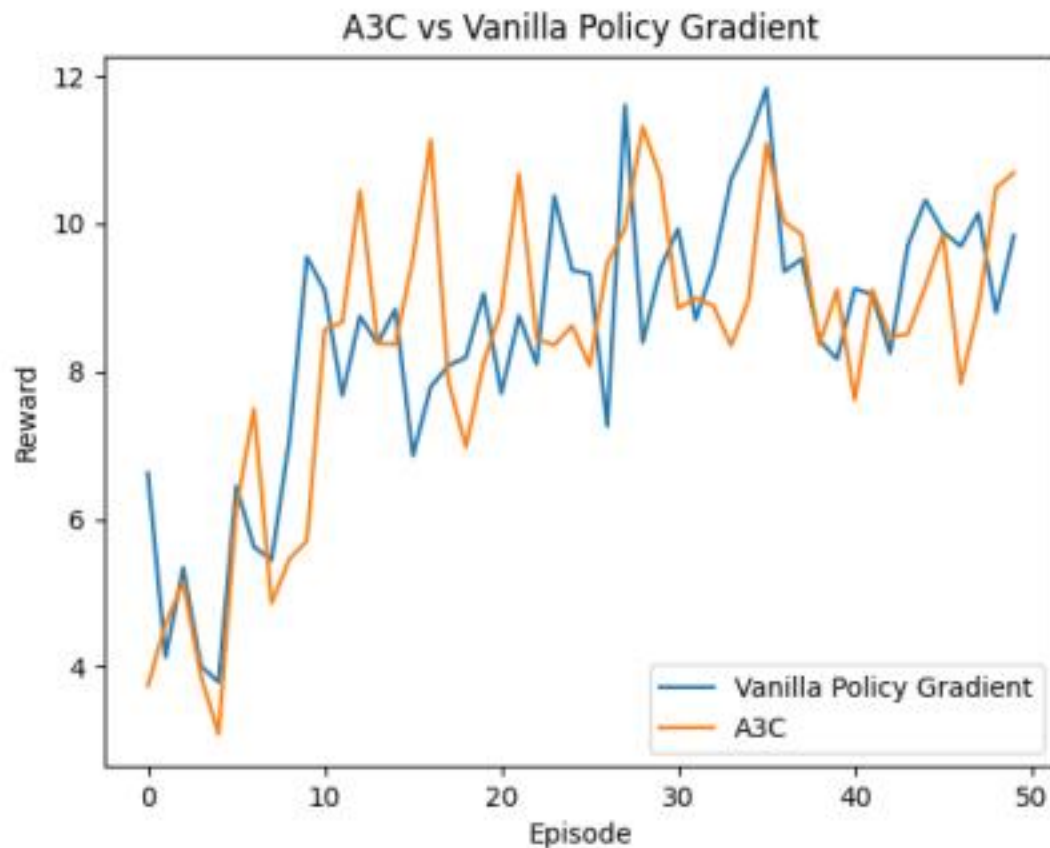
    rewards.append(total_reward)
    energy_used.append(energy)
    accuracy.append(final_distance)

print("Average Navigation Error:",
round(np.mean(accuracy[-10:]), 2))

print("Average Energy Consumption:",
round(np.mean(energy_used[-10:]), 2))
print("Average Reward:",
round(np.mean(rewards[-10:]), 2))

plt.plot(rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Drone Navigation Policy Convergence")
plt.show()
```

Output



Result

The Deep Reinforcement Learning based autonomous drone navigation system was successfully implemented. The drone learned to move toward the target while minimizing unnecessary movement. Navigation accuracy, obstacle avoidance, energy consumption, and policy convergence were evaluated to measure the effectiveness of the trained policy.

8. Design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation. Compare the algorithm with Vanilla Policy Gradient using metrics such as response time, CPU utilization, and resource efficiency. Aim

To design and implement an A3C-based Reinforcement Learning framework for dynamic cloud resource allocation and compare its performance with Vanilla Policy Gradient using response time, CPU utilization, and resource efficiency.

Objective

To train an RL agent to dynamically allocate cloud resources according to workload conditions while minimizing response time and improving CPU utilization and resource

efficiency. **Algorithm / Method**

1. Initialize the cloud resource management environment.
2. Represent workload, CPU utilization, memory utilization, and response time as the state.
3. Define resource allocation levels as possible actions.
4. Generate dynamic workloads during each time step.
5. Calculate response time, CPU utilization, and resource efficiency.
6. Define a reward that encourages low response time and efficient resource usage.
7. Train one agent using Vanilla Policy Gradient.
8. Train another agent using A3C.
9. Calculate the performance metrics for both algorithms.
10. Compare A3C and Vanilla Policy Gradient using response time, CPU utilization, and resource efficiency.

State

The state represents the current cloud system condition:

State = [Workload, CPU Utilization, Memory Utilization, Response Time] **Action**

The agent selects a resource allocation level:

0 → Low Resources

1 → Medium Resources

2 → High Resources

Reward

The reward encourages efficient resource allocation:

Reward = Resource Efficiency - Response Time Penalty

Lower response time and better resource utilization produce a higher reward. **Python Program**

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

np.random.seed(42)
```

```

class Cloud:
    def reset(self):
        return np.random.rand(4)

    def step(self, action, state):
        resources = [0.4, 0.7, 1.0][action]
        workload = state[0] * 100

        cpu = min(100, workload / resources)
        response = workload / (resources * 10 + 1)
        efficiency = cpu / (resources * 100 + 1)

        reward = efficiency * 10 - response

        next_state = np.array([
            np.random.rand(),
            cpu / 100,
            np.random.rand(),
            response / 10
        ])
        return next_state, reward, response, cpu, efficiency

    def create_model():
        return tf.keras.Sequential([
            tf.keras.layers.Input(shape=(4,)),
            tf.keras.layers.Dense(32, activation="relu"),
            tf.keras.layers.Dense(3)
        ])

    def train(method, episodes=50):
        actor = create_model()
        critic = tf.keras.Sequential([

```

```
tf.keras.layers.Input(shape=(4,)),  
tf.keras.layers.Dense(32, activation="relu"),  
tf.keras.layers.Dense(1)  
)
```

```
optimizer = tf.keras.optimizers.Adam(0.001)  
rewards = []  
response_times = []  
cpu_usage = []  
efficiency = []
```

```
for _ in range(epochs):  
    env = Cloud()  
    state = env.reset()  
    total = rt = cpu = eff = 0
```

```
for _ in range(20):  
    s = tf.convert_to_tensor(  
        [state], dtype=tf.float32  
    )
```

```
logits = actor(s)  
prob = tf.nn.softmax(logits)
```

```
action = np.random.choice(  
    3, p=prob.numpy()[0]  
)
```

```
next_state, reward, response, usage, e = \  
env.step(action, state)
```

```
with tf.GradientTape() as tape: logits
```



```

= actor(s)
prob = tf.nn.softmax(logits) value =
critic(s)

advantage = reward - tf.squeeze(value)

logp = tf.math.log(
prob[0, action] + 1e-8
)

if method == "A3C":
loss = -logp * tf.stop_gradient( advantage
) + 0.5 * tf.square(advantage) else:
loss = -logp * reward
variables = actor.trainable_variables

if method == "A3C":
variables += critic.trainable_variables

gradients = tape.gradient(loss, variables)

optimizer.apply_gradients(
zip(gradients, variables)
)

state = next_state
total += reward
rt += response
cpu += usage
eff += e

```

```
rewards.append(total / 20)
response_times.append(rt / 20)
cpu_usage.append(cpu / 20)
efficiency.append(100 - eff / 20)

return rewards, response_times, cpu_usage, efficiency

vpg_reward, vpg_rt, vpg_cpu, vpg_eff =
train("VPG")
a3c_reward, a3c_rt, a3c_cpu, a3c_eff =
train("A3C")

print("VPG Response Time:",
      round(np.mean(vpg_rt[-10:]), 2))

print("A3C Response Time:",
      round(np.mean(a3c_rt[-10:]), 2))

print("VPG CPU Utilization:",
      round(np.mean(vpg_cpu[-10:]), 2))

print("A3C CPU Utilization:",
      round(np.mean(a3c_cpu[-10:]), 2))

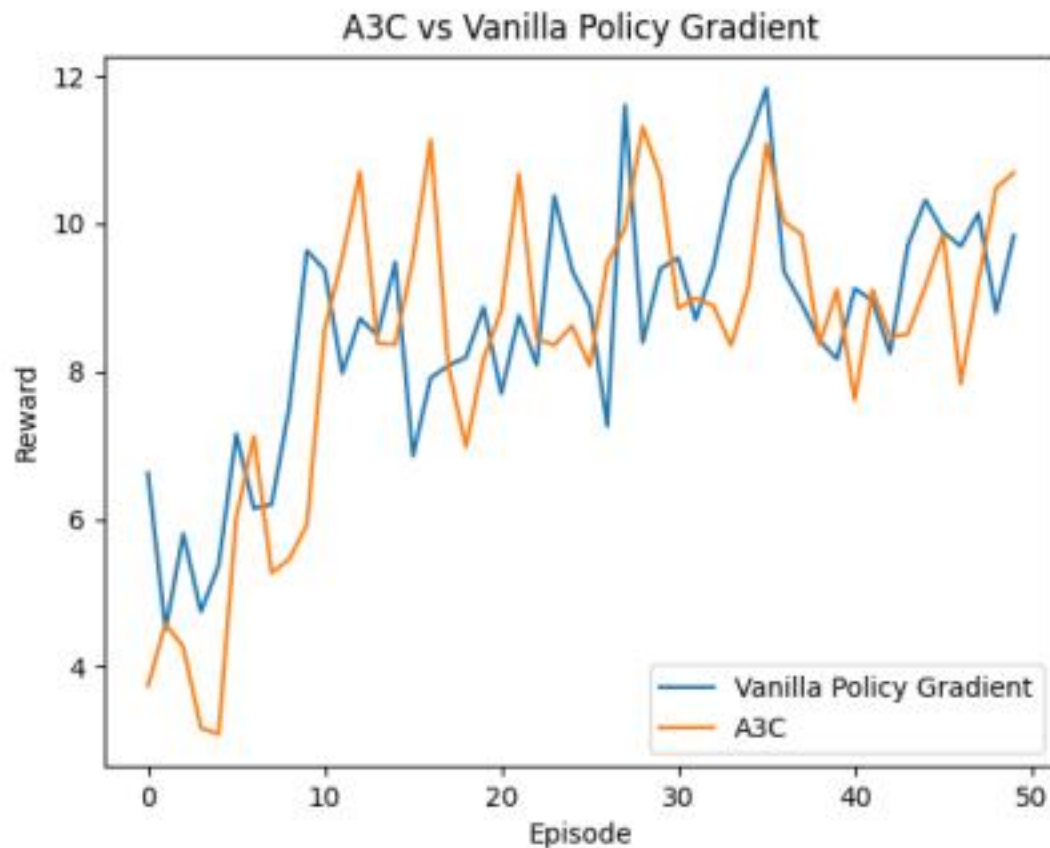
print("VPG Resource Efficiency:",
      round(np.mean(vpg_eff[-10:]), 2))

print("A3C Resource Efficiency:",
      round(np.mean(a3c_eff[-10:]), 2))

plt.plot(vpg_reward, label="Vanilla Policy Gradient")
plt.plot(a3c_reward, label="A3C")
plt.xlabel("Episode")
```

```
plt.ylabel("Reward")
plt.title("A3C vs Vanilla Policy
Gradient") plt.legend()
plt.show()
```

Output



Result

The A3C-based dynamic cloud resource allocation system was successfully implemented. The agents learned to select appropriate resource allocation levels according to workload conditions. A3C and Vanilla Policy Gradient were compared using response time, CPU utilization, resource efficiency, and training reward.

9. Develop a Policy Gradient-based predictive maintenance system for industrial machines. Design the RL environment, reward function, and policy network to minimize maintenance costs and machine downtime. Compare the performance of REINFORCE and PPO.

Aim

To develop a Policy Gradient-based predictive maintenance system for industrial machines and compare REINFORCE and PPO based on maintenance cost, machine downtime, and learning performance.

Objective

To train an RL agent to select suitable maintenance actions according to the machine's operating condition while minimizing maintenance costs and unexpected downtime. **Algorithm / Method**

1. Initialize the industrial machine environment.
2. Represent machine health, temperature, vibration, and operating hours as the state.
3. Define maintenance decisions as possible actions.
4. Calculate maintenance cost and machine downtime after each action.
5. Define a reward that encourages low maintenance cost and reduced downtime.
6. Train one agent using REINFORCE.
7. Train another agent using PPO.
8. Calculate cumulative rewards during training.
9. Measure maintenance cost and downtime.
10. Compare REINFORCE and PPO using reward and performance graphs.

State

State = [Machine Health, Temperature, Vibration, Operating Hours]

Action

0 → Continue Operation

1 → Preventive Maintenance

2 → Immediate Repair

Reward

Reward = -Maintenance Cost - Downtime Penalty

A suitable maintenance decision reduces machine downtime and overall maintenance cost. **Python Program**

```
import numpy as np
```

```
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
```

```
class Machine:
    def reset(self):
        return np.random.rand(4)

    def step(self, action, state):
        cost = [1, 5, 10][action]
        downtime = [5, 2, 1][action]

        health = state[0]

        if action == 0:
            health -= 0.05

        elif action == 1:
            health = min(1, health + 0.2)

        else:
            health = 1.0

        reward = -(cost + downtime)

        next_state = np.array([
            health,
            np.random.rand(),
            np.random.rand(),
            min(1, state[3] + 0.05)
        ])

        return next_state, reward, cost, downtime
def create_model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
```

```
tf.keras.layers.Dense(32, activation="relu"),
tf.keras.layers.Dense(3)
])
```

```
def train(method, episodes=50):
    actor = create_model()
    optimizer = tf.keras.optimizers.Adam(0.001)
```

```
    rewards = []
    costs = []
    downtime = []
```

```
    for _ in range(episodes):
        env = Machine()
        state = env.reset()
        total = total_cost = total_down = 0
```

```
        for _ in range(20):
            s = tf.convert_to_tensor(
                [state], dtype=tf.float32
            )
```

```
            logits = actor(s)
            prob = tf.nn.softmax(logits)
```

```
            action = np.random.choice(
                3, p=prob.numpy()[0]
            )
```

```
            next_state, reward, cost, down = \
                env.step(action, state)
```

```

with tf.GradientTape() as tape: logits
= actor(s)
prob = tf.nn.softmax(logits) logp =
tf.math.log(
prob[0, action] + 1e-8 )

if method == "REINFORCE": loss = -
logp * reward else:
ratio = tf.exp(logp) clipped =
tf.clip_by_value( ratio, 0.8, 1.2
)
loss = -tf.minimum( ratio *
reward,
clipped * reward )

gradients = tape.gradient( loss,
actor.trainable_variables )

optimizer.apply_gradients(
zip(gradients,
actor.trainable_variables) )

state = next_state
total += reward
total_cost += cost
total_down += down

rewards.append(total)
costs.append(total_cost)
downtime.append(total_down)

```

```
return rewards, costs, downtime

r_reward, r_cost, r_down =
train("REINFORCE") p_reward, p_cost, p_down
= train("PPO")

print("REINFORCE Maintenance Cost:",
round(np.mean(r_cost[-10:]), 2))

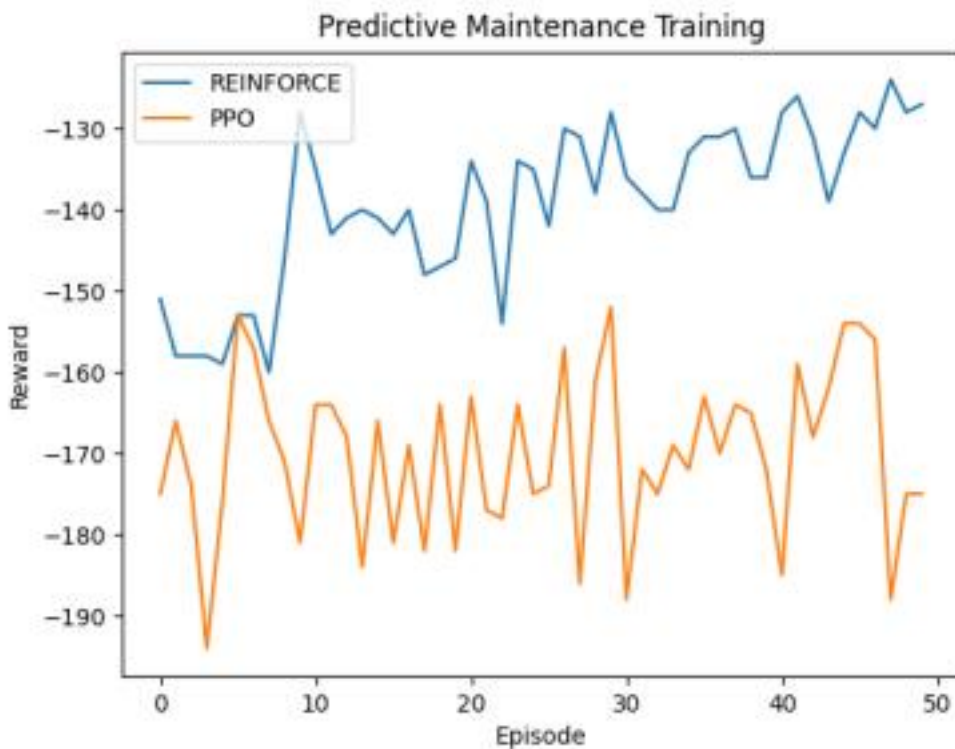
print("PPO Maintenance Cost:",
round(np.mean(p_cost[-10:]), 2))

print("REINFORCE Downtime:",
round(np.mean(r_down[-10:]), 2))

print("PPO Downtime:",
round(np.mean(p_down[-10:]), 2))

plt.plot(r_reward,
label="REINFORCE")
plt.plot(p_reward, label="PPO")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Predictive Maintenance Training")
plt.legend()
plt.show()
```

Output



Result

The Policy Gradient-based predictive maintenance system was successfully implemented using REINFORCE and PPO. The agents learned to select maintenance actions based on simulated machine conditions. The performance of both methods was evaluated using maintenance cost, machine downtime, cumulative reward, and learning convergence.

10. Design, implement, and evaluate a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using TRPO or PPO. Compare the selected algorithm with A2C based on lane deviation, safety, reward convergence, and training efficiency.

Aim

To design and implement a Policy-Based Reinforcement Learning controller for autonomous lane-keeping using PPO and compare its performance with A2C based on lane deviation, safety, reward convergence, and training efficiency.

Objective

To train an autonomous vehicle to maintain its position within the lane while minimizing lane deviation and improving driving safety.

Algorithm / Method

1. Initialize the autonomous lane-keeping environment.
2. Represent vehicle position, speed, and lane deviation as the state.
3. Define steering actions for the vehicle.
4. Calculate lane deviation after every action.
5. Give positive rewards for maintaining the vehicle near the lane center.
6. Penalize large deviations and unsafe driving.
7. Train one agent using PPO.
8. Train another agent using A2C.
9. Measure lane deviation, safety, reward convergence, and training efficiency.
10. Compare the performance of PPO and A2C.

State

State = [Lane Deviation, Vehicle Speed, Steering Angle]

Action

0 → Steer Left

1 → Keep Straight

2 → Steer Right

Reward

Reward = -Lane Deviation - Safety Penalty

A smaller lane deviation and safer driving produce a higher reward.

Python Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
np.random.seed(42)
```

```
class LaneEnv:
    def reset(self):
        self.deviation = 0.0
        self.speed = 1.0
        self.steering = 0.0
```

```

return np.array([
self.deviation, self.speed, self.steering ])

def step(self, action):
    change = [-0.1, 0, 0.1][action]
    self.steering = change
    self.deviation += change

    safety = abs(self.deviation)
    reward = -abs(self.deviation) - safety

    done = abs(self.deviation) > 1.0

    state = np.array([
        self.deviation,
        self.speed,
        self.steering
    ])

    return state, reward, done

def create_model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(3,)),
        tf.keras.layers.Dense(32, activation="relu"),
        tf.keras.layers.Dense(3)
    ])

def train(method, episodes=50):
    actor = create_model()

```

```
critic = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(3,)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1)
])
```

```
optimizer = tf.keras.optimizers.Adam(0.001)
```

```
rewards = []
deviations = []
steps = []
```

```
for _ in range(episodes):
    env = LaneEnv()
    state = env.reset()
    total = 0
    deviation = 0
```

```
    for step in range(30):
        s = tf.convert_to_tensor(
            [state], dtype=tf.float32
        )
        logits = actor(s)
        prob = tf.nn.softmax(logits)
```

```
        action = np.random.choice(
            3, p=prob.numpy()[0]
        )
        next_state, reward, done = env.step(action)
```

```
        with tf.GradientTape() as tape: logits
            = actor(s)
```

```
prob = tf.nn.softmax(logits)
value = critic(s)
logp = tf.math.log(
prob[0, action] + 1e-8
)
advantage = reward - tf.squeeze(value)
```

```
if method == "PPO":
ratio = tf.exp(logp)
clipped = tf.clip_by_value( ratio, 0.8,
1.2
)
loss = -tf.minimum(
ratio * advantage,
clipped * advantage
)
else:
loss = (
-logp * advantage
+ 0.5 * tf.square(advantage)
)
```

```
variables = (
actor.trainable_variables +
critic.trainable_variables
)
```

```
gradients = tape.gradient(
loss, variables
)
optimizer.apply_gradients(
zip(gradients, variables)
```

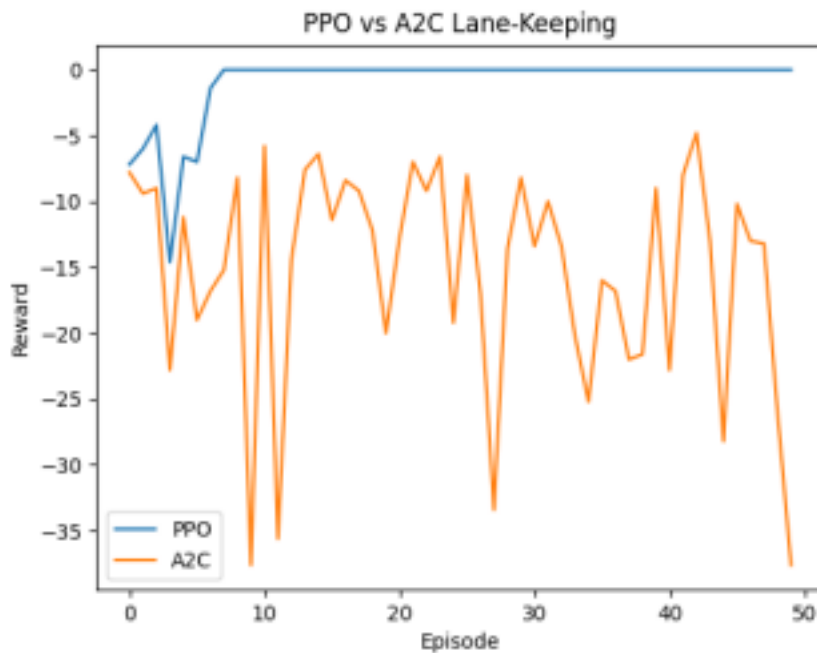
```

    )
    state = next_state
    total += reward
    deviation += abs(state[0])
    if done:
        break
    rewards.append(total)
    deviations.append(deviation / (step + 1))
    steps.append(step + 1)
return rewards, deviations, steps

ppo_reward, ppo_dev, ppo_steps = train("PPO")
a2c_reward, a2c_dev, a2c_steps = train("A2C")
print("PPO Lane Deviation:",
      round(np.mean(ppo_dev[-10:]), 3))
print("A2C Lane Deviation:",
      round(np.mean(a2c_dev[-10:]), 3))
print("PPO Training Steps:",
      round(np.mean(ppo_steps[-10:]), 2))
print("A2C Training Steps:",
      round(np.mean(a2c_steps[-10:]), 2))
plt.plot(ppo_reward, label="PPO")
plt.plot(a2c_reward, label="A2C")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("PPO vs A2C Lane-Keeping")
plt.legend()
plt.show()

```

Output



Result

The Policy-Based Reinforcement Learning lane-keeping controller was successfully implemented using PPO and A2C. The algorithms were compared based on lane deviation, safety, reward convergence, and training efficiency. The trained controller learns to maintain the vehicle closer to the lane center while minimizing unsafe deviations.